L12A: Metacircular Evaluator II

CS1101S: Programming Methodology

Martin Henz

4th November, 2026

Version 1.0 — Built: 2026-11-03 23:17:49 SGT

Recap: the evaluator so far

Applications and built-in functions

Adding functions

Local names

Running the evaluator

Lazy evaluation

Recap: the evaluator so far



```
def evaluate(component, env):
    if is_literal(component):
        return literal_value(component)
    elif is_name(component):
        return lookup_symbol_value(
            symbol_of_name(component), env)
    elif is_operator_combination(component):
        return eval_operator_combination(component, env)
    elif is_conditional(component):
        return eval_conditional(component, env)
    elif is_sequence(component):
        return eval_sequence(
            sequence_statements(component), env)
    elif is_assignment(component):
        return eval_assignment(component, env)
    else:
        return error('unknown syntax', component)
```

- parse turns program string into syntax tree
- Environments are linked lists of frames

Today

- Applications and built-in functions: `evaluate` meets `apply`
- Adding functions: definitions, lambda expressions, applications and return statements
- Local names, `global` and `nonlocal`
- Changing the language: lazy evaluation

Recap: the evaluator so far

Applications and built-in functions

Adding functions

Local names

Running the evaluator

Lazy evaluation

Built-in functions in the evaluator

```
built_in_functions = llist(
    llist('head', head),
    llist('tail', tail),
    llist('pair', pair),
    llist('is_none', is_none),
    llist('llist', llist),
    llist('print', print_without_environments),
    llist('error', error),
    llist('abs', abs),
    llist('+', lambda x, y: x + y),
    llist('-', lambda x, y: x - y),
    llist('*', lambda x, y: x * y),
```



Built-in functions are linked lists that start with the string `'built_in'`.

The global environment



```
built_in_function_symbols = map(head, built_in_functions)
built_in_function_objects = map(
    lambda fun: llist('built_in', head(tail(fun))),
    built_in_functions)

def setup_environment():
    built_in_environment = extend_environment(
        append(built_in_function_symbols,
               built_in_constant_symbols),
        append(built_in_function_objects,
               built_in_constant_values),
        the_empty_environment)
    return extend_environment(None, None,
                              built_in_environment)

the_global_environment = setup_environment()
```

Evaluating applications

In `evaluate`:

```
    elif is_application(component):
        return apply(
            evaluate(function_expression(component), env),
            llist_of_values(arg_expressions(component),
                            env))
    elif is_operator_combination(component):
        app = operator_combination_to_application(
            component)
        return evaluate(app, env)
```



The arguments are evaluated to a linked list of values:

```
def llist_of_values(exps, env):
    return map(lambda arg: evaluate(arg, env), exps)
```



Syntax functions: applications

```
def is_application(component):
    return is_tagged_llist(component, 'application')

def function_expression(component):
    return head(tail(component))

def arg_expressions(component):
    return head(tail(tail(component)))
```



Built-in functions

```
def is_built_in_function(fun):
    return is_tagged_llist(fun, 'built_in')

def built_in_implementation(fun):
    return head(tail(fun))

def apply_built_in_function(fun, arglist):
    return apply_in_underlying_python(
        built_in_implementation(fun), arglist)
```



apply applies a built-in function to the arguments:

```
def apply(fun, arguments):
    if is_built_in_function(fun):
        return apply_built_in_function(fun, arguments)
    else:
        return error('unknown function type', fun)
```



Example: applying a built-in function

```
tree = parse('abs(-5)')
env = the_global_environment
fun = evaluate(function_expression(tree), env)
args = llist_of_values(arg_expressions(tree), env)
print(is_built_in_function(fun))
print(apply(fun, args))
```



- `evaluate` finds the built-in function object of `abs` in the global environment
- `apply` applies Python's `abs` to the linked list of argument values

Operator combinations are applications

Operator combinations become applications of built-in functions:



```
def operator_combination_to_application(component):
    operator = operator_symbol(component)
    if is_unary_operator_combination(component):
        return make_application(
            make_name(operator),
            llist(first_operand(component)))
    else:
        return make_application(
            make_name(operator),
            llist(first_operand(component),
                  second_operand(component)))
```

Example: operator combinations

```
tree = parse('1 + 2 * 3')
app = operator_combination_to_application(tree)
print(app == llist('application',
                   llist('name', '+'),
                   llist(llist('literal', 1),
                         llist('binary_operator_combination',
                               '*', llist('literal', 2),
                               llist('literal', 3)))))
print(evaluate(tree, the_global_environment))
```



Only the outer combination is transformed; `2 * 3` is transformed when `evaluate` reaches it as an argument expression.

Recap: the evaluator so far

Applications and built-in functions

Adding functions

Local names

Running the evaluator

Lazy evaluation

Adding functions

stmt	::=	$\cdots$	
	|	`def` *name* `(` *params* `):` *stmts*	function definition
	|	`return` *expr*	return statement
expr	::=	$\cdots$	
	|	*expr*`(` $expr_1$`,` $\ldots$`,` $expr_n$`)`	application
	|	`lambda` *params* `:` *expr*	lambda

Example:

```
set_evaluator(parse_and_evaluate)
set_program_text("""
def fact(n):
    return 1 if n == 1 else n * fact(n - 1)
print(fact(5))
""")
```



Parsing function definitions

`parse('def f(x):\n    return x + 1')` returns

```
tree = llist('function_definition',
             llist('name', 'f'),
             llist(llist('name', 'x')),
             llist('return_statement',
                   llist('binary_operator_combination',
                         '+',
                         llist('name', 'x'),
                         llist('literal', 1))))
print(parse('def f(x):\n    return x + 1') == tree)
```



- Syntax predicate: `is_function_definition`
- Selectors: `function_definition_symbol`, `function_definition_parameter_symbols`, `function_definition_body`

Parsing lambda expressions and applications

`parse('lambda x: x * 2')` and `parse('f(1, 2)')` return



```
tree1 = llist('lambda_expression',
              llist(llist('name', 'x')),
              llist('binary_operator_combination',
                    '*',
                    llist('name', 'x'),
                    llist('literal', 2)))
tree2 = llist('application',
              llist('name', 'f'),
              llist(llist('literal', 1),
                    llist('literal', 2)))
print(parse('lambda x: x * 2') == tree1)
print(parse('f(1, 2)') == tree2)
```

- Syntax predicate: `is_lambda_expression`
- Selectors: `lambda_parameter_symbols`, `lambda_body`

Syntax functions: function definitions



```
def is_function_definition(component):
    return is_tagged_llist(component, 'function_definition')

def function_definition_name(component):
    return llist_ref(component, 1)

def function_definition_symbol(component):
    return symbol_of_name(
        function_definition_name(component))

def function_definition_parameters(component):
    return llist_ref(component, 2)

def function_definition_parameter_symbols(component):
    return map(symbol_of_name,
               function_definition_parameters(component))

def function_definition_body(component):
    return llist_ref(component, 3)
```

Syntax functions: lambda, return



```
def is_lambda_expression(component):
    return is_tagged_llist(component,
                           'lambda_expression')
def lambda_parameter_symbols(component):
    return map(symbol_of_name, head(tail(component)))
def lambda_body(component):
    return head(tail(tail(component)))
def is_return_statement(component):
    return is_tagged_llist(component,
                           'return_statement')
def return_expression(component):
    return head(tail(component))
def make_return_statement(expression):
    return llist('return_statement', expression)
```

Evaluating functions

In `evaluate`:



```
    elif is_lambda_expression(component):
        return make_function(
            lambda_parameter_symbols(component),
            None, None,
            make_return_statement(lambda_body(component)),
            env)
    elif is_function_definition(component):
        return eval_function_definition(component, env)
    elif is_return_statement(component):
        return eval_return_statement(component, env)
```

- A lambda expression makes a function whose body is a single return statement; it cannot contain assignments or declarations, so it has no local or global names (`None`)
- A function definition makes a function and assigns it to the name
- A return statement evaluates to a *return value*

Function objects



```
def make_function(parameters, locals, globals,
                  body, env):
    return llist('compound_function', parameters,
                 locals, globals, body, env)

def function_parameters(f):
    return llist_ref(f, 1)
def function_locals(f):
    return llist_ref(f, 2)
def function_globals(f):
    return llist_ref(f, 3)
def function_body(f):
    return llist_ref(f, 4)
def function_environment(f):
    return llist_ref(f, 5)
```

- A function object carries its local names and its global names
- A function definition determines them; see Section “Local names”

Return statements

The Problems

- Functions that do not evaluate a return statement must return `None`
- Evaluation of a return statement *anywhere* in the function body will return from the function with the result of evaluating the return expression

Return statements: an example

```
set_evaluator(parse_and_evaluate)
set_program_text("""
def f(n, b):
    if b:
        return n
    return -n
print(f(5, True))
print(f(5, False))
""")
```



Representing return values

Evaluating a return statement wraps the value in a *return value*:



```
def eval_return_statement(component, env):
    return make_return_value(
        evaluate(return_expression(component), env))

def make_return_value(content):
    return llist('return_value', content)

def is_return_value(value):
    return is_tagged_llist(value, 'return_value')

def return_value_content(value):
    return head(tail(value))
```

Return values in sequences



```
def eval_sequence(stmts, env):
    if is_empty_sequence(stmts):
        return None
    elif is_last_statement(stmts):
        return evaluate(first_statement(stmts), env)
    else:
        first_stmt_value = evaluate(
            first_statement(stmts), env)
        if is_return_value(first_stmt_value):
            return first_stmt_value
        else:
            return eval_sequence(
                rest_statements(stmts), env)
```

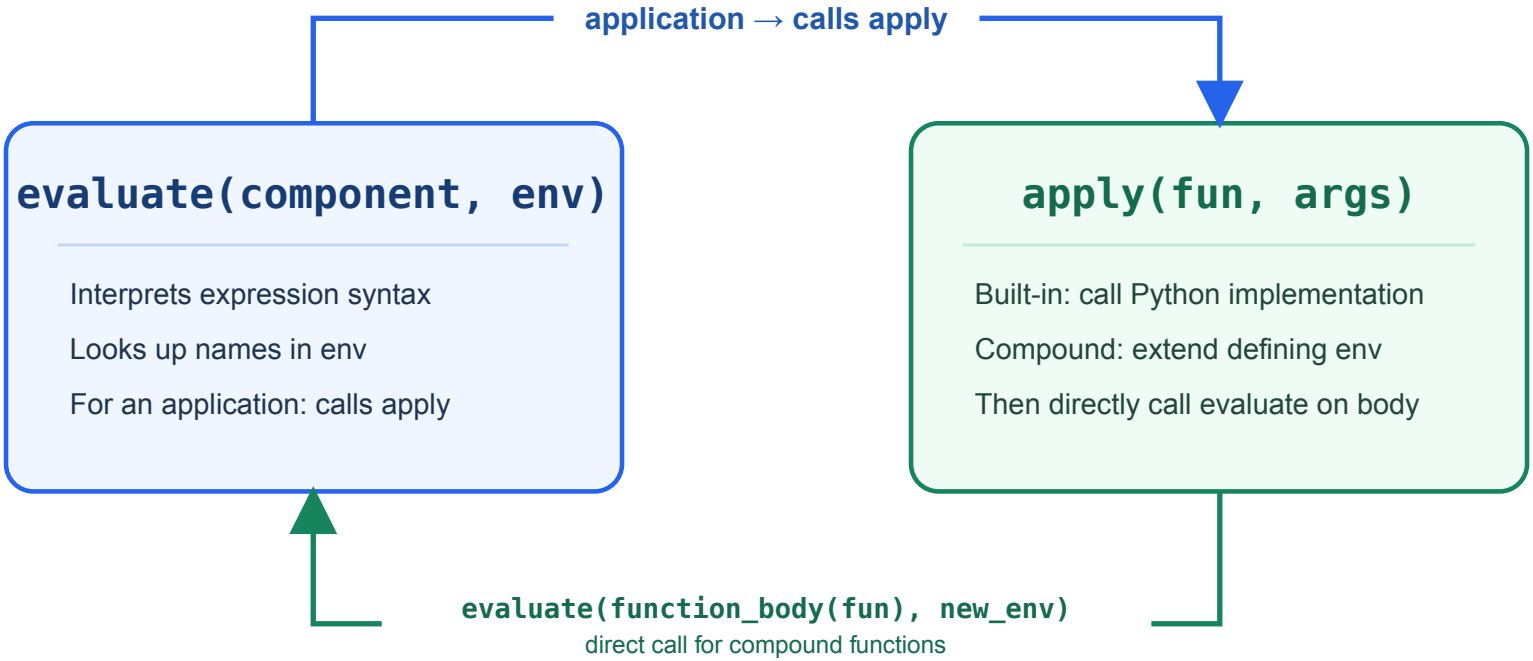
Applying compound functions (first try)



```
def apply(fun, arguments):
    if is_built_in_function(fun):
        return apply_built_in_function(fun, arguments)
    elif is_compound_function(fun):
        result = evaluate(
            function_body(fun),
            extend_environment(
                function_parameters(fun),
                arguments,
                function_environment(fun)))
        if is_return_value(result):
            return return_value_content(result)
        else:
            return None
    else:
        return error('unknown function type', fun)
```

- The body is evaluated in a new frame for the parameters
- Only `apply` unwraps a return value; no `return` gives `None`
- For now, we ignore local and global names

The evaluate–apply loop



Recap: the evaluator so far

Applications and built-in functions

Adding functions

Local names

Running the evaluator

Lazy evaluation

Names in function bodies

```
set_evaluator(parse_and_evaluate)
set_program_text("""
x = 1
def g():
    x = 2
    return x
print(g())
print(x)
""")
```



- An assignment in a function body makes the name *local* to the function
- `global x`: the name refers to the global frame
- `nonlocal x`: the name refers to an enclosing function's frame

Function definitions



```
def eval_function_definition(component, env):
    parameters = function_definition_parameter_symbols(
        component)
    body = function_definition_body(component)
    fun = make_function(parameters,
                        local_variables(parameters, body),
                        global_variables(body),
                        body, env)
    assign_symbol_value(
        function_definition_symbol(component), fun, env)
    return None
def local_variables(parameters, body):
    excluded = append(parameters,
                      append(global_variables(body),
                             nonlocal_variables(body)))
    return filter(
        lambda s: is_none(member(s, excluded)),
        assigned_variables(body))
```

- Global names: declared by `global` statements in the body
- Local names: assigned in body (not `global`/`nonlocal`)

Applying compound functions (complete)

```
def apply(fun, arguments):
    if is_built_in_function(fun):
        return apply_built_in_function(fun, arguments)
    elif is_compound_function(fun):
        locals = function_locals(fun)
        globals = function_globals(fun)
        unassigned_markers = map(
            lambda s: '*unassigned*', locals)
        global_markers = map(
            lambda s: '*global*', globals)
```



- The function object supplies the local and global names
- Local names start out `'*unassigned*'`; global names are marked `'*global*'`

Applying compound functions (cont.)



```
        new_env = extend_environment(
            append(function_parameters(fun),
                   append(locals, globals)),
            append(arguments,
                   append(unassigned_markers,
                          global_markers)),
            function_environment(fun))
        result = evaluate(function_body(fun), new_env)
        if is_return_value(result):
            return return_value_content(result)
        else:
            return None
    else:
        return error('unknown function type', fun)
```

Names marked `*global*`

In `lookup_symbol_value`:

```
            elif symbol == head(symbols):
                if head(vals) == '*global*':
                    return lookup_symbol_value(
                        symbol, the_global_environment)
                else:
                    return head(vals)
```



In `assign_symbol_value`:

```
        elif symbol == head(symbols):
            if head(vals) == '*global*':
                assign_symbol_value(
                    symbol, val, the_global_environment)
            else:
                set_head(vals, val)
```



`nonlocal` names are simply not made local, so that the search continues in the enclosing environment.

Recap: the evaluator so far

Applications and built-in functions

Adding functions

Local names

Running the evaluator

Lazy evaluation

Running the evaluator



```
from repl import set_evaluator, set_program_text

def parse_and_evaluate(program):
    evaluate(parse(program), the_global_environment)

set_evaluator(parse_and_evaluate)
set_program_text("""
def f(n, b):
    if b:
        return n
    return -n
x = 1
def g():
    x = 2
    return x
print(f(5, False))
print(g())
print(x)
""")
```

The Repl tab calls `parse_and_evaluate` with every program that you run there; programs print with `print`.

Recap: the evaluator so far

Applications and built-in functions

Adding functions

Local names

Running the evaluator

Lazy evaluation

An example

```
set_evaluator(parse_and_evaluate)
set_program_text("""
def unless(condition, usual, exceptional):
    return exceptional if condition else usual
xs = None
unless(is_none(xs), head(xs), print("xs is empty"))
""")
```



Can we implement `unless` like this? No: Python evaluates all three arguments *before* calling `unless`, so `head(xs)` fails when `xs` is empty.

Open question

Python evaluates function arguments before calling the function. Must a language work this way?

Recall: applicative and normal order (Lecture L2A)

Applicative order:

```
->  sum_of_squares(5 + 1, 5 * 2)
->  sum_of_squares(6, 10)
->  square(6) + square(10)
```

Normal order:

```
->  sum_of_squares(5 + 1, 5 * 2)
->  square(5 + 1) + square(5 * 2)
->  ((5 + 1) * (5 + 1)) + ((5 * 2) * (5 * 2))
```

Python, like our evaluator so far, uses applicative order.

Lazy evaluation

Laziness

In applications, arguments are not evaluated. They are passed to the function as expressions.

Force evaluation when needed

The argument expressions are evaluated when needed, during evaluation of the body

Non-strict and strict

Compound functions are non-strict in each argument; built-in functions stay strict.

Implementation

"Eager" evaluation (applicative order)

```
    elif is_application(component):
        return apply(
            evaluate(function_expression(component), env),
            llist_of_values(arg_expressions(component),
                            env))
```



"Lazy" evaluation (normal order)

```
    elif is_application(component):
        return apply(
            actual_value(
                function_expression(component), env),
            arg_expressions(component),
            env)
```



Modifying `apply`

```
def apply(fun, args, env):
    if is_built_in_function(fun):
        return apply_built_in_function(
            fun, llist_of_arg_values(args, env))
    elif is_compound_function(fun):
        delayed_args = llist_of_delayed_args(args, env)
```



- Built-in functions are strict: their arguments are evaluated
- Compound functions are non-strict: their arguments are delayed

Argument evaluation

```
def actual_value(exp, env):
    return force_it(evaluate(exp, env))

def llist_of_arg_values(exps, env):
    return map(lambda exp: actual_value(exp, env), exps)

def llist_of_delayed_args(exps, env):
    return map(lambda exp: delay_it(exp, env), exps)
```



A conditional needs the actual value of its predicate:

```
def eval_conditional(component, env):
    predicate = conditional_predicate(component)
    if is_truthy(actual_value(predicate, env)):
        return evaluate(
            conditional_consequent(component), env)
    else:
        return evaluate(
            conditional_alternative(component), env)
```



Implementation of thunks



```
def delay_it(exp, env):
    return llist('thunk', exp, env)

def is_thunk(obj):
    return is_tagged_llist(obj, 'thunk')

def thunk_exp(thunk):
    return head(tail(thunk))

def thunk_env(thunk):
    return head(tail(tail(thunk)))

def is_evaluated_thunk(obj):
    return is_tagged_llist(obj, 'evaluated_thunk')

def thunk_value(evaluated_thunk):
    return head(tail(evaluated_thunk))
```

Implementation of forcing (simple version)

```
def force_it(obj):
    if is_thunk(obj):
        return actual_value(thunk_exp(obj),
                            thunk_env(obj))
    else:
        return obj
```



Implementation of forcing (with memoization)



```
def force_it(obj):
    if is_thunk(obj):
        result = actual_value(thunk_exp(obj),
                              thunk_env(obj))
        set_head(obj, 'evaluated_thunk')
        set_head(tail(obj), result)  # store value
        set_tail(tail(obj), None)    # drop env
        return result
    elif is_evaluated_thunk(obj):
        return thunk_value(obj)
    else:
        return obj
```

Built-in functions stay strict

```
set_evaluator(parse_and_evaluate)
set_program_text("""
def make_ints(n):
    return pair(n, make_ints(n + 1))
ints = make_ints(1)
""")
```



- `make_ints` is compound, but `pair` is built-in
- `pair` evaluates its arguments: `make_ints(1)` needs `make_ints(2)`, which needs `make_ints(3)`, ...

Lazy pairs



```
set_evaluator(parse_and_evaluate)
set_program_text("""
def pair(x, y):
    return lambda m: m(x, y)
def head(z):
    return z(lambda p, q: p)
def tail(z):
    return z(lambda p, q: q)
def make_ints(n):
    return pair(n, make_ints(n + 1))
ints = make_ints(1)
print(head(tail(ints)))
""")
```

- Compound `pair`, `head` and `tail` shadow the built-in ones
- Compound functions are non-strict, so these pairs are lazy: infinite linked lists for free

Running the lazy evaluator



```
from repl import set_evaluator, set_program_text

def parse_and_evaluate(program):
    actual_value(parse(program), the_global_environment)

set_evaluator(parse_and_evaluate)
set_program_text("""
def unless(condition, usual, exceptional):
    return exceptional if condition else usual
xs = None
unless(is_none(xs), head(xs), print("xs is empty"))
""")
```

`head(xs)` is never evaluated: the argument `usual` is never forced.

Summary

- Metacircular evaluator:
  the most beautiful program ever written (?)
- Explains the environment model
- `evaluate`—`apply` loop
- Alternative understanding of `return`
- Local and global names: determined by function definitions, marked by `apply` in the new frame
- Lazy evaluation: delay arguments as thunks, force them when needed
- What other rules of a programming language could we change?

Check-In time!

Don't be lazy, but do think about it—or should we say, *thunk* about it!

Look for Check-In I12A in the Source Academy!